JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class LogManager

java.lang.Object
    java.util.logging.LogManager

```
public class LogManager
extends Object
```

There is a single global LogManager object that is used to maintain a set of shared state about Loggers and log services.

This LogManager object:

- Manages a hierarchical namespace of Logger objects. All named Loggers are stored in this namespace.
- Manages a set of logging control properties. These are simple key-value pairs that can be used by Handlers and other logging objects to configure themselves.

The global LogManager object can be retrieved using LogManager.getLogManager(). The LogManager object is created during class initialization and cannot subsequently be changed.

At startup the LogManager class is located using the java.util.logging.manager system property.

LogManager Configuration

A LogManager initializes the logging configuration via the `readConfiguration()` method during LogManager initialization. By default, LogManager default configuration is used. The logging configuration read by LogManager must be in the properties file format.

The LogManager defines two optional system properties that allow control over the initial configuration, as specified in the `readConfiguration()` method:

- `java.util.logging.config.class`
- `java.util.logging.config.file`

These two system properties may be specified on the command line to the "java" command, or as system property definitions passed to JNI_CreateJavaVM.

The properties for loggers and Handlers will have names starting with the dot-separated name for the handler or logger.
The global logging properties may include:

- A property "handlers". This defines a whitespace or comma separated list of class names for handler classes to load and register as handlers on the

root Logger (the Logger named ""). Each class name must be for a Handler class which has a default constructor. Note that these Handlers may be created lazily, when they are first used.

- A property "<logger>.handlers". This defines a whitespace or comma separated list of class names for handlers classes to load and register as handlers to the specified logger. Each class name must be for a Handler class which has a default constructor. Note that these Handlers may be created lazily, when they are first used.
- A property "<logger>.handlers.ensureCloseOnReset". This defines a a boolean value. If "<logger>.handlers" is not defined or is empty, this property is ignored. Otherwise it defaults to `true`. When the value is `true`, the handlers associated with the logger are guaranteed to be closed on reset() and shutdown. This can be turned off by explicitly setting "<logger>.handlers.ensureCloseOnReset=false" in the configuration. Note that turning this property off causes the risk of introducing a resource leak, as the logger may get garbage collected before `reset()` is called, thus preventing its handlers from being closed on `reset()`. In that case it is the responsibility of the application to ensure that the handlers are closed before the logger is garbage collected.
- A property "<logger>.useParentHandlers". This defines a boolean value. By default every logger calls its parent in addition to handling the logging message itself, this often result in messages being handled by the root logger as well. When setting this property to false a Handler needs to be configured for this logger otherwise no logging messages are delivered.
- A property "config". This property is intended to allow arbitrary configuration code to be run. The property defines a whitespace or comma separated list of class names. A new instance will be created for each named class. The default constructor of each class may execute arbitrary code to update the logging configuration, such as setting logger levels, adding handlers, adding filters, etc.

Note that all classes loaded during LogManager configuration are first searched on the system class path before any user class path. That includes the LogManager class, any config classes, and any handler classes.

Loggers are organized into a naming hierarchy based on their dot separated names. Thus "a.b.c" is a child of "a.b", but "a.b1" and a.b2" are peers.

All properties whose names end with ".level" are assumed to define log levels for Loggers. Thus "foo.level" defines a log level for the logger called "foo" and (recursively) for any of its children in the naming hierarchy. Log Levels are applied in the order they are defined in the properties file. Thus level settings for child nodes in the tree should come after settings for their parents. The property name ".level" can be used to set the level for the root of the tree.

All methods on the LogManager object are multi-thread safe.

Since:

1.4

Field Summary

Fields

Modifier and Type	Field	Description
`static final String`	`LOGGING_MXBEAN_NAME`	String representation of the `ObjectName` for the management interface for the logging facility.

Constructor Summary

Constructors

Modifier	Constructor	Description
`protected`	`LogManager()`	Protected constructor.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
`LogManager`	`addConfigurationListener(Runnable listener)`	Adds a configuration listener to be invoked each time the logging configuration is read.
`boolean`	`addLogger(Logger logger)`	Add a named logger.
`void`	`checkAccess()`	**Deprecated, for removal: This API element is subject to removal in a future version.** This method originally checked that the current context was trusted to modify the logging configuration.

Logger	**getLogger**(String name)	Method to find a named logger.
Enumeration<String>	**getLoggerNames**()	Get an enumeration of known logger names.
static LoggingMXBean	**getLoggingMXBean**()	**Deprecated.** java.util.logging.LoggingMXBean is deprecated and replaced with java.lang.management.PlatformLogging
static LogManager	**getLogManager**()	Returns the global LogManager object.
String	**getProperty** (String name)	Get the value of a logging property.
void	**readConfiguration**()	Reads and initializes the logging configuration.
void	**readConfiguration** (InputStream ins)	Reads and initializes the logging configuration from the given input stream.
void	**removeConfigurationListe** (Runnable listener)	Removes a previously registered configuration listener.
void	**reset**()	Reset the logging configuration.
void	**updateConfiguration** (InputStream ins, Function<String, BiFunction<String, String,String>> mapper)	Updates the logging configuration.
void	**updateConfiguration** (Function<String, BiFunction<String, String,String>> mapper)	Updates the logging configuration.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Details

LOGGING_MXBEAN_NAME

```
public static final String LOGGING_MXBEAN_NAME
```

String representation of the ObjectName for the management interface for the logging facility.

Since:

1.5

See Also:

PlatformLoggingMXBean, Constant Field Values

Constructor Details

LogManager

```
protected LogManager()
```

Protected constructor. This is protected so that container applications (such as J2EE containers) can subclass the object. It is non-public as it is intended that there only be one LogManager object, whose value is retrieved by calling LogManager.getLogManager.

Method Details

getLogManager

```
public static LogManager getLogManager()
```

Returns the global LogManager object.

Returns:

the global LogManager object

addLogger

```
public boolean addLogger(Logger logger)
```

Add a named logger. This does nothing and returns false if a logger with the same name is already registered.

The Logger factory methods call this method to register each newly created Logger.

The application should retain its own reference to the Logger object to avoid it being garbage collected. The LogManager may only retain a weak reference.

Parameters:

`logger` - the new logger.

Returns:

true if the argument logger was registered successfully, false if a logger of that name already exists.

Throws:

`NullPointerException` - if the logger name is null.

getLogger

```
public Logger getLogger(String name)
```

Method to find a named logger.

Note that since untrusted code may create loggers with arbitrary names this method should not be relied on to find Loggers for security sensitive logging. It is also important to note that the Logger associated with the String `name` may be garbage collected at any time if there is no strong reference to the Logger. The caller of this method must check the return value for null in order to properly handle the case where the Logger has been garbage collected.

Parameters:

`name` - name of the logger

Returns:

matching logger or null if none is found

getLoggerNames

```
public Enumeration<String> getLoggerNames()
```

Get an enumeration of known logger names.

Note: Loggers may be added dynamically as new classes are loaded. This method only reports on the loggers that are currently registered. It is also important to note that this method only returns the name of a Logger, not a strong reference to the Logger itself. The returned String does nothing to

prevent the Logger from being garbage collected. In particular, if the returned name is passed to `LogManager.getLogger()`, then the caller must check the return value from `LogManager.getLogger()` for null to properly handle the case where the Logger has been garbage collected in the time since its name was returned by this method.

Returns:

enumeration of logger name strings

readConfiguration

```
public void readConfiguration()
                       throws IOException
```

Reads and initializes the logging configuration.

If the "java.util.logging.config.class" system property is set, then the property value is treated as a class name. The given class will be loaded, an object will be instantiated, and that object's constructor is responsible for reading in the initial configuration. (That object may use other system properties to control its configuration.) The alternate configuration class can use `readConfiguration(InputStream)` to define properties in the LogManager.

If "java.util.logging.config.class" system property is **not** set, then this method will read the initial configuration from a properties file and calls the `readConfiguration(InputStream)` method to initialize the configuration. The "java.util.logging.config.file" system property can be used to specify the properties file that will be read as the initial configuration; if not set, then the LogManager default configuration is used. The default configuration is typically loaded from the properties file "`conf/logging.properties`" in the Java installation directory.

Any registered configuration listener will be invoked after the properties are read.

API Note:

This `readConfiguration` method should only be used for initializing the configuration during LogManager initialization or used with the "java.util.logging.config.class" property. When this method is called after loggers have been created, and the "java.util.logging.config.class" system property is not set, all existing loggers will be reset. Then any existing loggers that have a level property specified in the new configuration stream will be set to the specified log level.

To properly update the logging configuration, use the `updateConfiguration(java.util.function.Function)` or

updateConfiguration(java.io.InputStream, java.util.function.Function) methods instead.

Throws:

IOException - if there are IO problems reading the configuration.

reset

```
public void reset()
```

Reset the logging configuration.

For all named loggers, the reset operation removes and closes all Handlers and (except for the root logger) sets the level to null. The root logger's level is set to Level.INFO.

API Note:

Calling this method also clears the LogManager properties. The updateConfiguration(Function) or updateConfiguration(InputStream, Function) method can be used to properly update to a new configuration.

readConfiguration

```
public void readConfiguration(InputStream ins)
                       throws IOException
```

Reads and initializes the logging configuration from the given input stream.

Any registered configuration listener will be invoked after the properties are read.

API Note:

This readConfiguration method should only be used for initializing the configuration during LogManager initialization or used with the "java.util.logging.config.class" property. When this method is called after loggers have been created, all existing loggers will be reset. Then any existing loggers that have a level property specified in the given input stream will be set to the specified log level.

To properly update the logging configuration, use the updateConfiguration(java.util.function.Function) or updateConfiguration(java.io.InputStream, java.util.function.Function) method instead.

Parameters:

ins - stream to read properties from

Throws:

IOException - if there are problems reading from the stream, or the given stream is not in the properties file format.

updateConfiguration

```
public void updateConfiguration
(Function<String, BiFunction<String,String,String>> mapper)
                          throws IOException
```

Updates the logging configuration.

If the "java.util.logging.config.file" system property is set, then the property value specifies the properties file to be read as the new configuration. Otherwise, the LogManager default configuration is used.
The default configuration is typically loaded from the properties file "`conf/logging.properties`" in the Java installation directory.

This method reads the new configuration and calls the `updateConfiguration(ins, mapper)` method to update the configuration.

API Note:

This method updates the logging configuration from reading a properties file and ignores the "java.util.logging.config.class" system property. The "java.util.logging.config.class" property is only used by the `readConfiguration()` method to load a custom configuration class as an initial configuration.

Parameters:

`mapper` - a functional interface that takes a configuration key *k* and returns a function *f(o,n)* whose returned value will be applied to the resulting configuration. The function *f* may return `null` to indicate that the property *k* will not be added to the resulting configuration.
If `mapper` is `null` then `(k) -> ((o, n) -> n)` is assumed.
For each *k*, the mapped function *f* will be invoked with the value associated with *k* in the old configuration (i.e *o*) and the value associated with *k* in the new configuration (i.e. *n*).
A `null` value for *o* or *n* indicates that no value was present for *k* in the corresponding configuration.

Throws:

`NullPointerException` - if `mapper` returns a `null` function when invoked.

`IOException` - if there are problems reading from the logging configuration file.

Since:

9

See Also:

`updateConfiguration(java.io.InputStream, java.util.function.Function)`

updateConfiguration

```
public void updateConfiguration
(InputStream ins,
 Function<String, BiFunction<String,String,String>> mapper)
                          throws IOException
```

Updates the logging configuration.

For each configuration key in the existing configuration and the given input stream configuration, the given `mapper` function is invoked to map from the configuration key to a function, *f(o,n)*, that takes the old value and new value and returns the resulting value to be applied in the resulting configuration, as specified in the table below.

Let *k* be a configuration key in the old or new configuration, *o* be the old value (i.e. the value associated with *k* in the old configuration), *n* be the new value (i.e. the value associated with *k* in the new configuration), and *f* be the function returned by `mapper.apply(k)`: then *v* = *f(o,n)* is the resulting value. If *v* is not `null`, then a property *k* with value *v* will be added to the resulting configuration. Otherwise, it will be omitted.
A `null` value may be passed to function *f* to indicate that the corresponding configuration has no configuration key *k*. The function *f* may return `null` to indicate that there will be no value associated with *k* in the resulting configuration.

If `mapper` is `null`, then *v* will be set to *n*.

LogManager properties are updated with the resulting value in the resulting configuration.

The registered configuration listeners will be invoked after the configuration is successfully updated.

Property	Resulting Behavior
`<logger>.level`	• If the resulting configuration defines a level for a logger and if the resulting level is different than the level specified in the the old configuration, or not specified in the old configuration, then if the logger exists or if children for that logger exist, the level for that

Property	Resulting Behavior
	logger will be updated, and the change propagated to any existing logger children. This may cause the logger to be created, if necessary. • If the old configuration defined a level for a logger, and the resulting configuration doesn't, then this change will not be propagated to existing loggers, if any. To completely replace a configuration - the caller should therefore call `reset` to empty the current configuration, before calling `updateConfiguration`.
`<logger>.useParentHandlers`	• If either the resulting or the old value for the useParentHandlers property is not null, then if the logger exists or if children for that logger exist, that logger will be updated to the resulting value. The value of the useParentHandlers property is the value specified in the configuration; if not specified, the default is true.
`<logger>.handlers`	• If the resulting configuration defines a list of handlers for a logger, and if the resulting list is different than the list specified in the old configuration for that logger (that could be empty), then if the logger exists or its children exist, the handlers associated with that logger are closed and removed and the new handlers will be created per the resulting configuration and added to that logger, creating that logger if necessary. • If the old configuration defined some handlers for a logger, and the resulting configuration doesn't, if that logger exists, its handlers will be removed and closed. • Changing the list of handlers on an existing logger will cause all its previous handlers to be removed and closed, regardless of whether they had

Property	Resulting Behavior
	been created from the configuration or programmatically. The old handlers will be replaced by new handlers, if any.
`<handler-name>.*`	• Properties configured/changed on handler classes will only affect newly created handlers. If a node is configured with the same list of handlers in the old and the resulting configuration, then these handlers will remain unchanged.
`config` and any other property	• The resulting value for these property will be stored in the LogManager properties, but `updateConfiguration` will not parse or process their values.

Example mapper functions:

- Replace all logging properties with the new configuration:

  `(k) -> ((o, n) -> n)`:

  this is equivalent to passing a null `mapper` parameter.
- Merge the new configuration and old configuration and use the new value if *k* exists in the new configuration:

  `(k) -> ((o, n) -> n == null ? o : n)`:

  as if merging two collections as follows: `result.putAll(oldc); result.putAll(newc)`.
- Merge the new configuration and old configuration and use the old value if *k* exists in the old configuration:

  `(k) -> ((o, n) -> o == null ? n : o)`:

  as if merging two collections as follows: `result.putAll(newc); result.putAll(oldc)`.
- Replace all properties with the new configuration except the handler property to configure Logger's handler that is not root logger:

```
(k) -> k.endsWith(".handlers")
         ? ((o, n) -> (o == null ? n : o))
         : ((o, n) -> n)
```

To completely reinitialize a configuration, an application can first call `reset` to fully remove the old configuration, followed by `updateConfiguration` to initialize the new configuration.

Parameters:

`ins` - a stream to read properties from

`mapper` - a functional interface that takes a configuration key *k* and returns a function *f(o,n)* whose returned value will be applied to the resulting configuration. The function *f* may return `null` to indicate that the property *k* will not be added to the resulting configuration.
If `mapper` is `null` then `(k) -> ((o, n) -> n)` is assumed.
For each *k*, the mapped function *f* will be invoked with the value associated with *k* in the old configuration (i.e *o*) and the value associated with *k* in the new configuration (i.e. *n*).
A `null` value for *o* or *n* indicates that no value was present for *k* in the corresponding configuration.

Throws:

`NullPointerException` - if `ins` is null or if `mapper` returns a null function when invoked.

`IOException` - if there are problems reading from the stream, or the given stream is not in the properties file format.

Since:

9

getProperty

```
public String getProperty(String name)
```

Get the value of a logging property. The method returns null if the property is not found.

Parameters:

`name` - property name

Returns:

property value

checkAccess

```
@Deprecated(since="17",
            forRemoval=true)
public void checkAccess()
```

Deprecated, for removal: This API element is subject to removal in a future version.
This method originally checked that the current context was trusted to modify the logging configuration. This method was only useful in conjunction with the Security Manager, which is no longer supported. There is no replacement for the Security Manager or this method.

Does nothing.

getLoggingMXBean

```
@Deprecated(since="9")
public static LoggingMXBean getLoggingMXBean()
```

Deprecated.
`java.util.logging.LoggingMXBean` is deprecated and replaced with `java.lang.management.PlatformLoggingMXBean`. Use `ManagementFactory.getPlatformMXBean`(PlatformLoggingMXBean.class) instead.

Returns `LoggingMXBean` for managing loggers.

Returns:

a `LoggingMXBean` object.

Since:

1.5

See Also:

`PlatformLoggingMXBean`

addConfigurationListener

```
public LogManager addConfigurationListener(Runnable listener)
```

Adds a configuration listener to be invoked each time the logging configuration is read. If the listener is already registered the method does nothing.

The listener is invoked with privileges that are restricted by the calling context of this method. The order in which the listeners are invoked is unspecified.

It is recommended that listeners do not throw errors or exceptions. If a listener terminates with an uncaught error or exception then the first exception will be propagated to the caller of `readConfiguration()` (or `readConfiguration(java.io.InputStream)`) after all listeners have been invoked.

Implementation Note:

If more than one listener terminates with an uncaught error or exception, an implementation may record the additional errors or exceptions as suppressed exceptions.

Parameters:

`listener` - A configuration listener that will be invoked after the configuration changed.

Returns:

This LogManager.

Throws:

`NullPointerException` - if the listener is null.

Since:

9

removeConfigurationListener

```
public void removeConfigurationListener(Runnable listener)
```

Removes a previously registered configuration listener. Returns silently if the listener is not found.

Parameters:

`listener` - the configuration listener to remove.

Throws:

`NullPointerException` - if the listener is null.

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.